WEEK 6

NAME : A. JEEVITESWARA REDDY
ROLL NO : CH.SC.U4CSE24164

Huffman coding:

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_TREE_HT 100
#define ASCII 256

typedef struct Node {
    char data;
    int freq;
    struct Node *left, *right;
} Node;

typedef struct {
    int size;
    Node* arr[ASCII];
} MinHeap;

Node* newNode(char data, int freq) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->data = data;
    n->freq = freq;
    n->left = n->right = NULL;
    return n;
}

MinHeap* createHeap() {
    MinHeap* h = (MinHeap*)malloc(sizeof(MinHeap));
    h->size = 0;
    return h;
}

void swap(Node** a, Node** b) {
    Node* t = *a; *a = *b; *b = t;
}

void heapify(MinHeap* h, int i) {
    int smallest = i;
    int l = 2*i+1, r = 2*i+2;

    if (l < h->size && h->arr[l]->freq < h->arr[smallest]->freq)
```

```
GNU nano 7.2
    if (l < h->size && h->arr[l]->freq < h->arr[smallest]->freq)
        smallest = l;
    if (r < h->size && h->arr[r]->freq < h->arr[smallest]->freq)
        smallest = r;

    if (smallest != i) {
        swap(&h->arr[i], &h->arr[smallest]);
        heapify(h, smallest);
    }
}

void insertHeap(MinHeap* h, Node* n) {
    int i = h->size++;
    while (i && n->freq < h->arr[(i-1)/2]->freq) {
        h->arr[i] = h->arr[(i-1)/2];
        i = (i-1)/2;
    }
    h->arr[i] = n;
}

Node* extractMin(MinHeap* h) {
    Node* temp = h->arr[0];
    h->arr[0] = h->arr[--h->size];
    heapify(h, 0);
    return temp;
}

Node* buildHuffman(char* text) {
    int freq[ASCII] = {0};
    for (int i=0; text[i]; i++)
        freq[(unsigned char)text[i]]++;

    MinHeap* h = createHeap();

    for (int i=0; i<ASCII; i++)
        if (freq[i])
            insertHeap(h, newNode(i, freq[i]));

    while (h->size > 1) {
        Node* left = extractMin(h);
        Node* right = extractMin(h);
```

```c
    while (h->size > 1) {
        Node* left = extractMin(h);
        Node* right = extractMin(h);
        Node* top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        insertHeap(h, top);
    }
    return extractMin(h);
}

void generateCodes(Node* root, char* code, int depth, char codes[ASCII][MAX_TREE_HT]) {
    if (!root) return;

    if (!root->left && !root->right) {
        code[depth] = '\0';
        strcpy(codes[(unsigned char)root->data], code);
    }

    code[depth] = '0';
    generateCodes(root->left, code, depth+1, codes);

    code[depth] = '1';
    generateCodes(root->right, code, depth+1, codes);
}

void decode(Node* root, char* encoded) {
    Node* curr = root;
    printf("Decoded: ");

    for (int i=0; encoded[i]; i++) {
        curr = (encoded[i] == '0') ? curr->left : curr->right;
        if (!curr->left && !curr->right) {
            printf("%c", curr->data);
            curr = root;
        }
    }
    printf("\n");
}
```

```c
int main() {
    char text[1000];

    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text,"\n")] = 0;

    Node* root = buildHuffman(text);

    char codes[ASCII][MAX_TREE_HT] = {{0}};
    char code[MAX_TREE_HT];
    generateCodes(root, code, 0, codes);

    printf("\nCharacter Codes:\n");
    for (int i=0;i<ASCII;i++)
        if (codes[i][0])
            printf("%c : %s\n", i, codes[i]);

    printf("\nEncoded: ");
    char encoded[10000]="";
    for (int i=0; text[i]; i++) {
        printf("%s", codes[(unsigned char)text[i]]);
        strcat(encoded, codes[(unsigned char)text[i]]);
    }

    printf("\n");
    decode(root, encoded);

    return 0;
}
```

OUTPUT:

```
Enter text: data analysis and intelligence laboratory

Character Codes:
  : 001
a : 110
b : 100110
c : 100111
d : 0000
e : 1110
g : 10010
i : 1010
l : 010
n : 011
o : 0001
r : 11110
s : 11111
t : 1011
y : 1000

Encoded: 0000110101111000111001111001010001111110101111100111001100000011010011101111100100101010100101110011100111111000101011010011000011111011010110001111101000
Decoded: data analysis and intelligence laboratory
```

WORKING:

Huffman coding

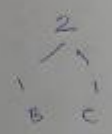
Data Analysis and Intelligence Laboratory

D A T N L Y S I E G C B O R

2 7 3 4 4 2 2 3 3 1 1 1 2 2

B C G D O R S Y E I T L N A

1 1 1 2 2 2 2 2 3 3 3 4 4 7

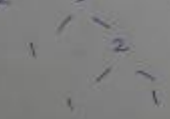


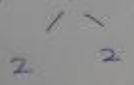
1 2 2 2 2 2 2 3 3 3 4 4 7

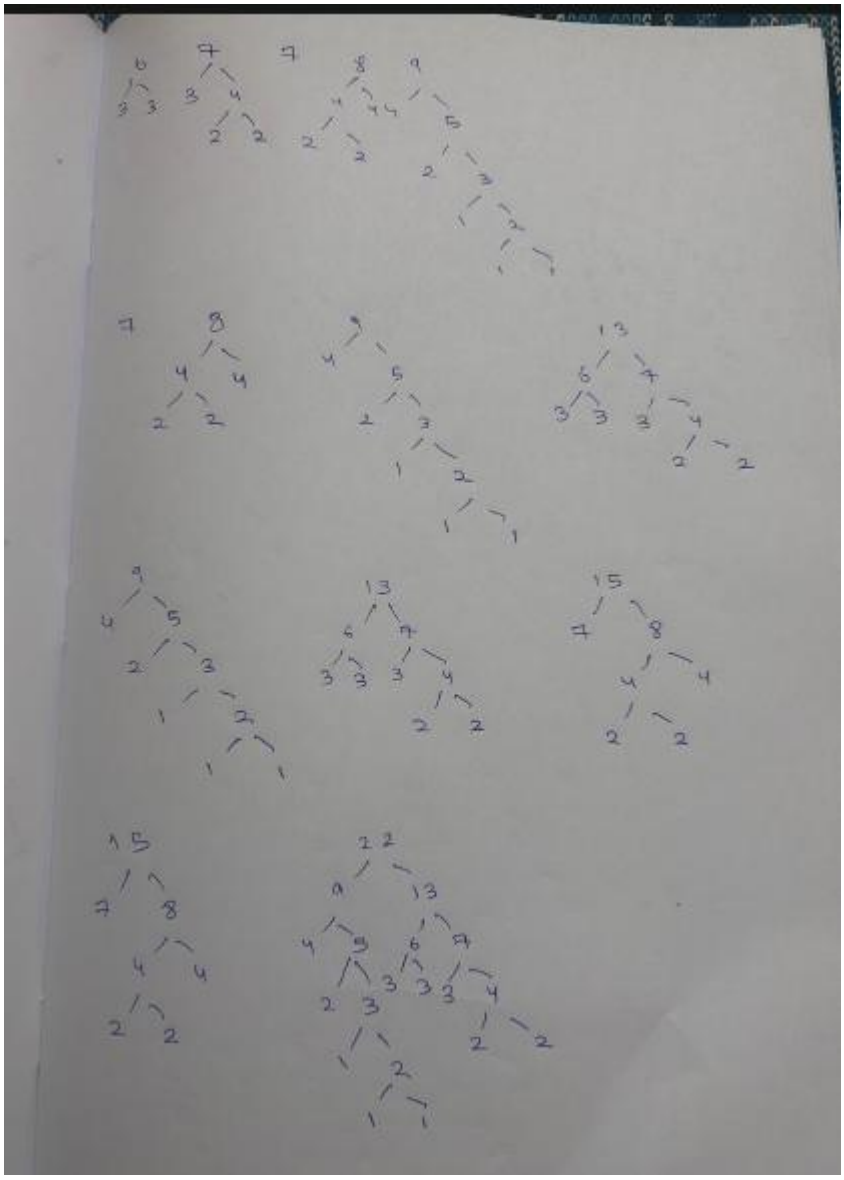
G D O R S Y E I T L N A

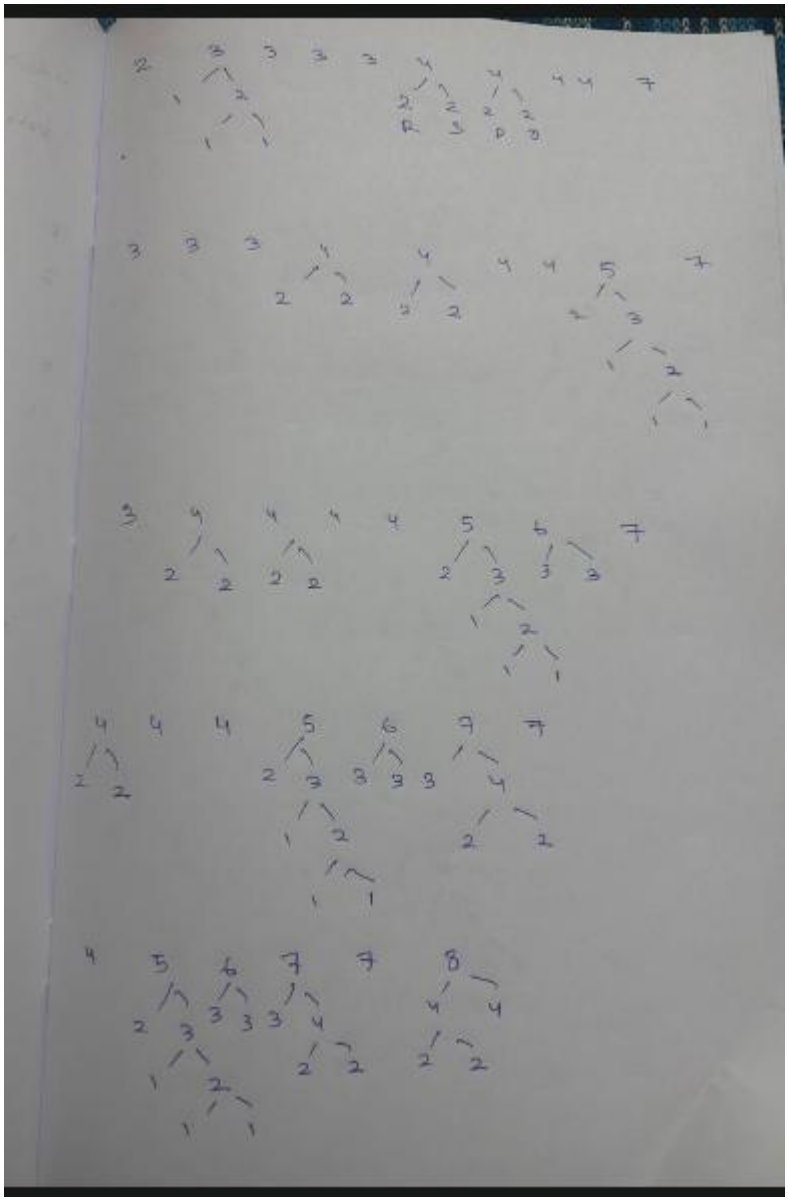
2 2 2 2 2 3 3 3 3 4 4 7

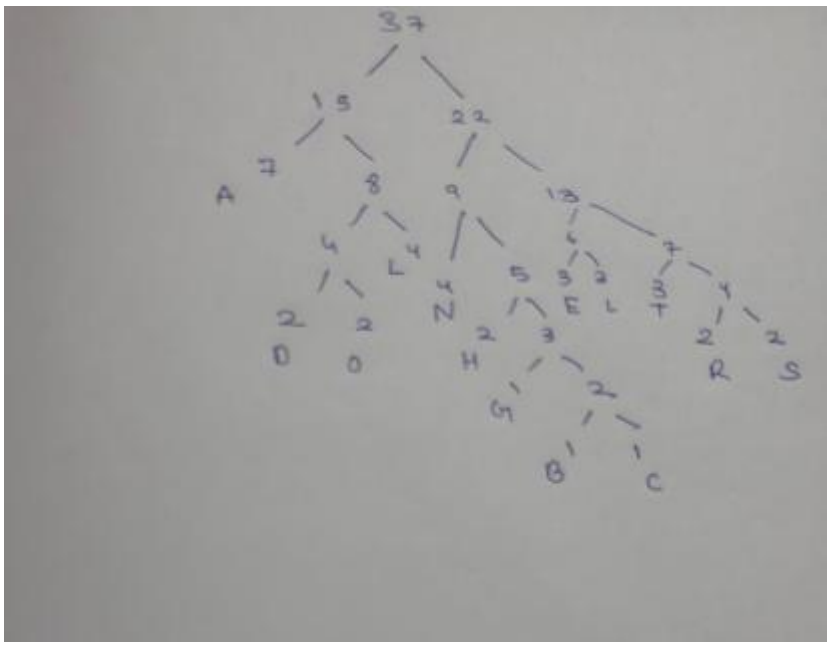
2 2 2 3 3 3 3 4 4 4 7










37
15
22
7
A
8
9
13
4
4
4
5
6
7
2
2
N
2
3
3
2
3
4
D
O
H
E
L
T
R
S
G
2
B
C